



University of Zurich^{UZH}

High Performance Computing Lecture 7

Douglas Potter

Cesare Cozza

Mark Eberlein

Parallel Programming: an introduction

History

- Moore's law

Parallel processing

- Amdahl's and Gustafson's laws

Hardware evolution

- Vector, parallel and accelerated processors

Programming languages

- MPI, OpenMP, CUDA, OpenACC

Moore's law and hardware constraints



Gordon Moore: founder of Intel.

Statement of the law:

The number of transistors that can be placed on an integrated circuit at a reasonable cost doubles every two years.

Energy consumption and cost:

- Dissipated electric power scales at clock frequency to the cube.
- Dissipated power by square cm is limited by cooling

Constraints:

- Processor frequency reached a plateau at around 3GHz since 2002-2004 but Moore's law is still true
- More cores per chip (many core architectures, GPU...)

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Transistor count

50,000,000,000

10,000,000,000

5,000,000,000

1,000,000,000

500,000,000

100,000,000

50,000,000

10,000,000

5,000,000

1,000,000

500,000

100,000

50,000

10,000

5,000

1,000

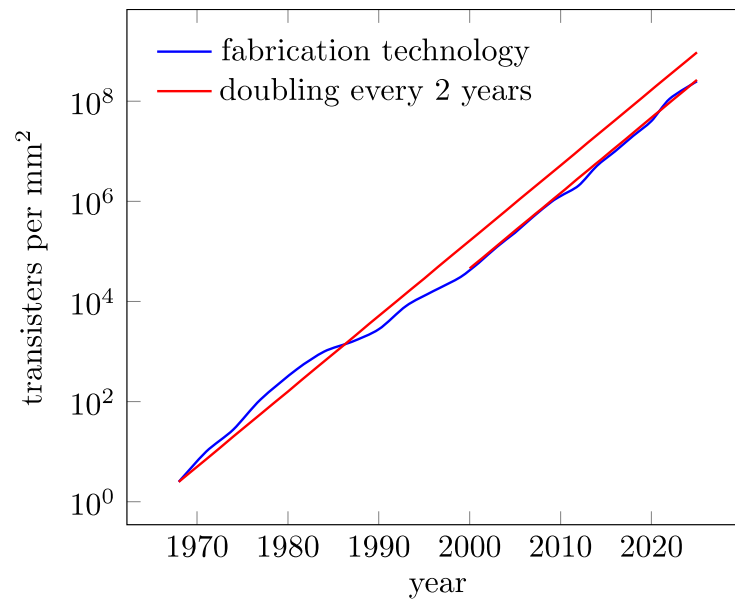
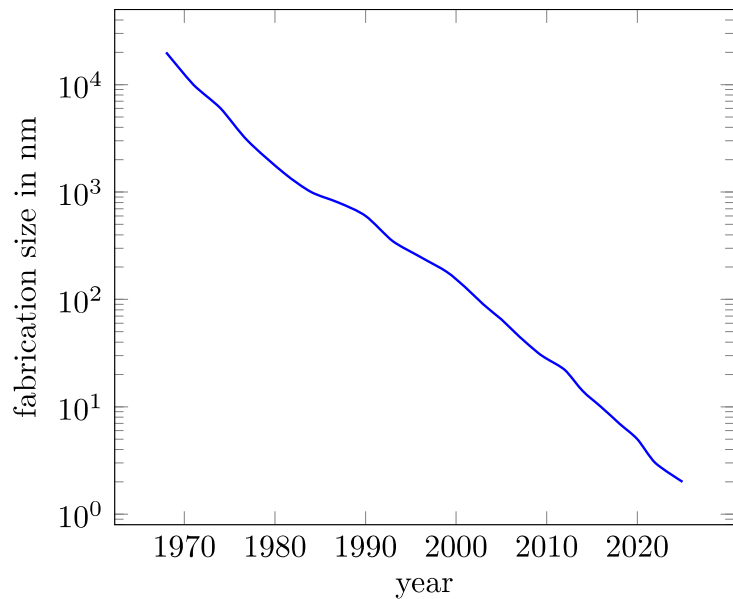
Year in which the microchip was first introduced

Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

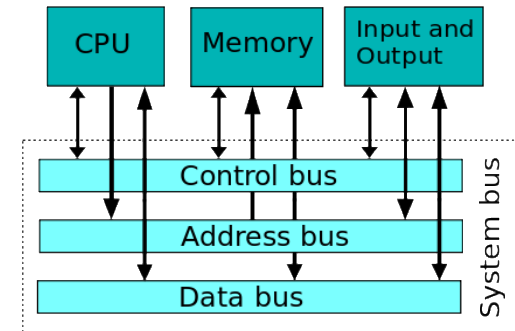
Fabrication Size



The Memory Wall

Von Neumann Architecture

Programming instructions (stored-program) and data share the memory unit. They are loaded to the Central Processing Unit (CPU) for execution. Results are loaded back to the memory unit.



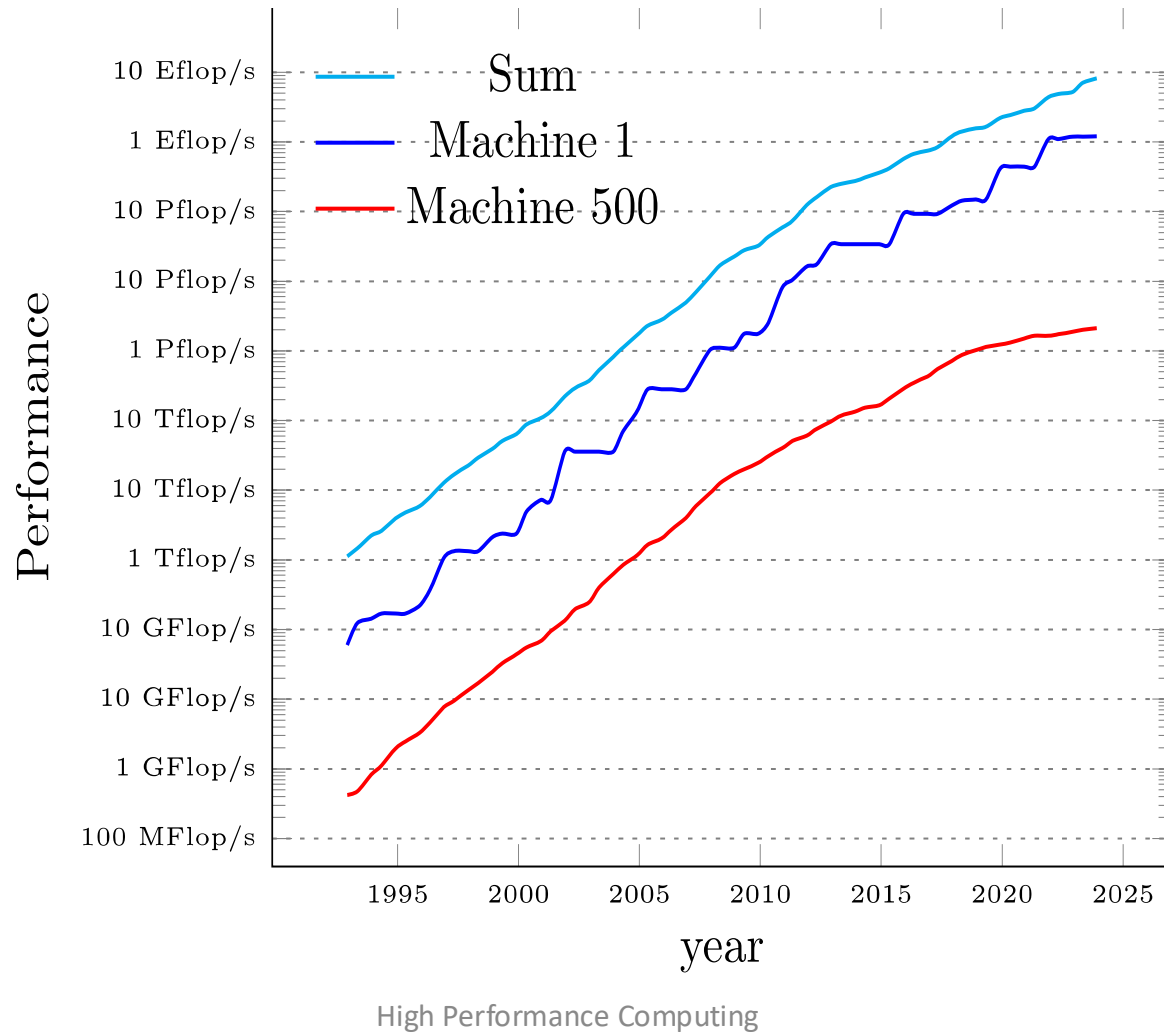
The Von Neumann Bottleneck:

- Memory bandwidth is not increasing as quickly as processor computing power
- Memory latency is decreasing very slowly
- Number of computing cores per memory units is increasing

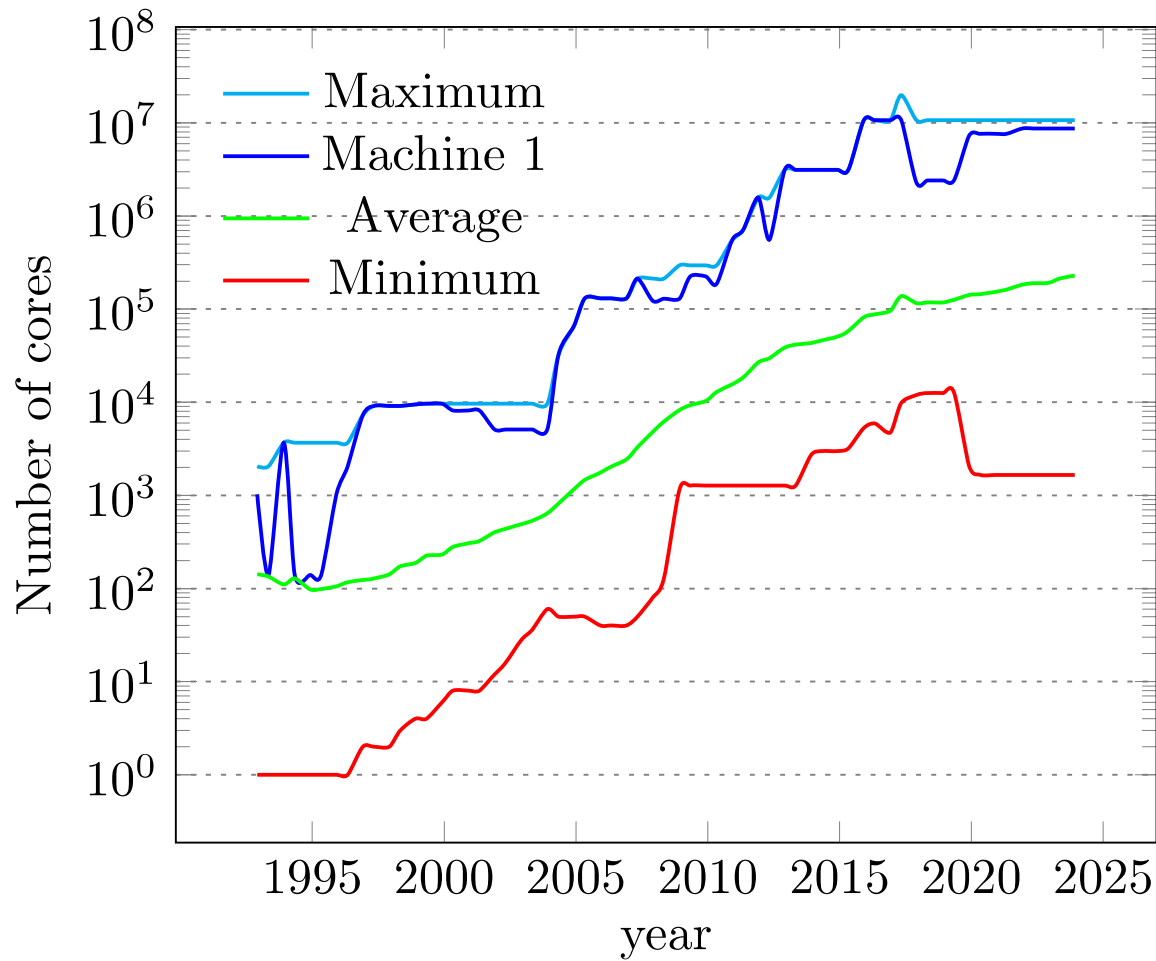
Consequences and solutions:

- The CPU wastes cycles while waiting for data.
- Introduction of cache memory (L1, L2, L3)
- Parallel access to memory (vector architectures, AVX)

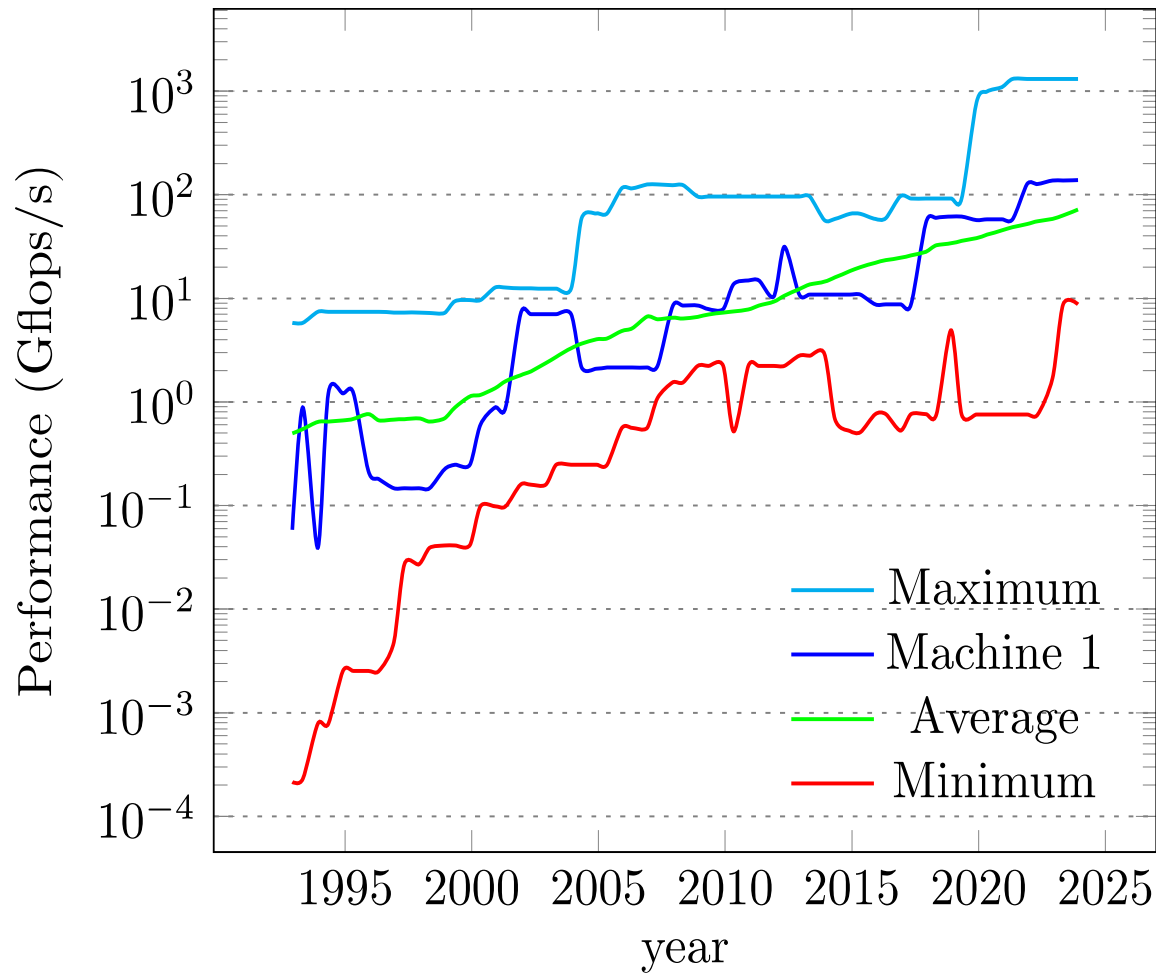
Performance Development



Evolution of
the number
of cores on
Top500



Evolution of
the
performance
per core on
Top500





Parallel Processing

Gene Amdahl (1967): Amdahl's Law

Statement of the law:

The theoretical maximum speedup obtained by parallelizing a code ideally, *for a given problem with a fixed size*:

$$\text{Speedup}(N) = \frac{T_s}{T_p(N)} = \frac{T_s}{\alpha T_s + (1-\alpha)\frac{T_s}{N}} = \frac{1}{\alpha + \frac{1-\alpha}{N}}$$

T_s : execution time of the serial code

T_p : execution time of the parallel code

α : fraction of the code that is not parallel

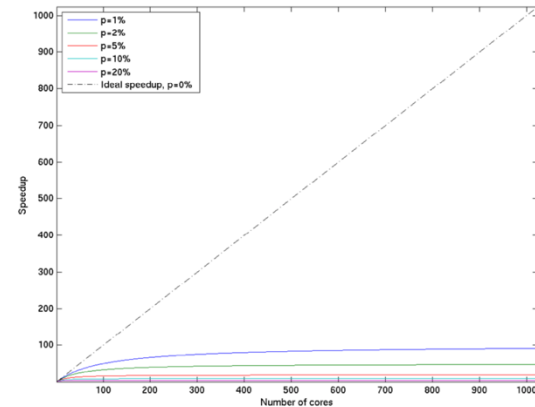
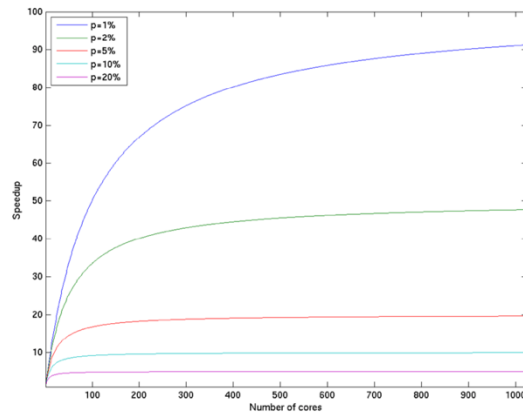
N : number of processors

$$\text{Speedup}(N) \rightarrow \frac{1}{\alpha} \text{ as } N \rightarrow +\infty$$

Strong scaling

Theoretical Maximum Speedup

Cores	α (%)								
	0	0.01	0.1	1	2	5	10	25	50
10	10	9.99	9.91	9.17	8.47	6.90	5.26	3.08	1.82
100	100	99.0	91.0	50.2	33.6	16.8	9.17	3.88	1.98
1000	1000	909	500	91	47.7	19.6	9.91	3.99	1.998
10000	10000	5000	909	99.0	49.8	19.96	9.99	3.99	2
100000	100000	9091	990	99.9	49.9	19.99	10	4	2
∞	∞	10000	1000	100	50	20	10	4	2





John Gustafson and Edwin Barsis (1988): Gustafson's Law

Statement of the law:

The theoretical maximum speedup obtained by parallelizing a code ideally *for a problem of constant size per core*:

$$\text{Speedup}(N) = \frac{T_S}{T_P(N)} = \alpha + (1 - \alpha)N$$

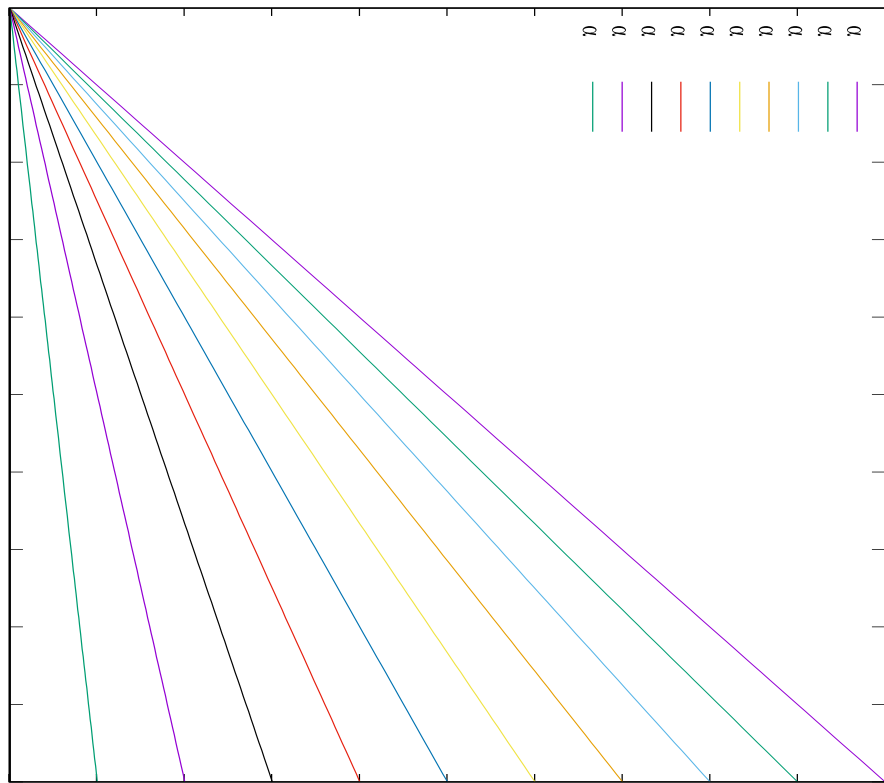
Basic premise: **with an already parallelized task, what is the theoretical “slowdown” of the task if run serially?**

This law is more optimistic than Amdahl's law:

$$\text{Speedup}(N) \rightarrow (1 - \alpha)N \text{ as } N \rightarrow +\infty$$

In both laws, everything is determined by α , the non-parallel fraction of the code. Reducing α is called “parallel code optimization”.

Weak scaling



Strong versus Weak scaling

Original problem size:

- 1 billion elements on a single core

Option 1:

- 1 billion elements using 100 cores
- 10 million elements per core

Option 2:

- 100 billion elements using 100 cores
- 1 billion elements per core

Which is “strong” and “weak” scaling?

Strong versus Weak scaling

Original problem size:

- 1 billion elements on a single core

Strong Scaling (same problem size):

- 1 billion elements using 100 cores
- 10 million elements per core

Weak scaling (same size **per core**):

- 100 billion elements using 100 cores
- 1 billion elements per core

Which is “strong” and “weak” scaling?

Hardware Evolution

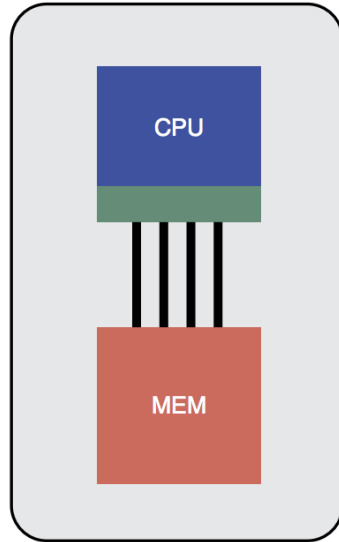
Technical trends:

- Computing power is doubling every year
- Massively parallel and many-cores architectures are dominant
- Since 2010, Graphical Processing Units become a key player
- Increasing hardware complexity (hybrid system, multiple cache layers)
- Memory per core is stagnating or decreasing
- Performance per core is increasing slowly or stagnating
- Disk Input/Output bandwidth is increasing very slowly

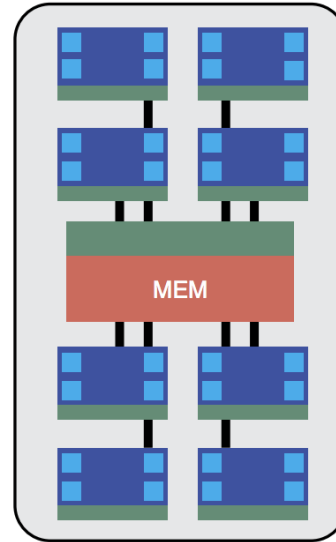


Cray XC50 with GPU

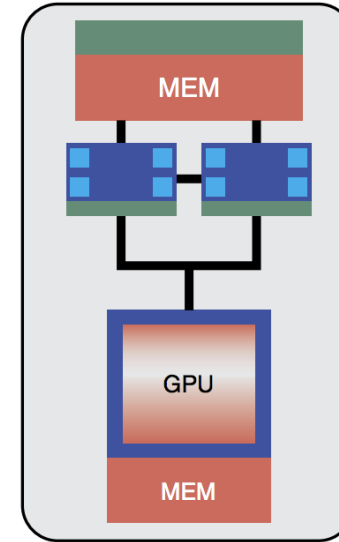
Evolution of supercomputers



1995
Single CPU per node with
main memory

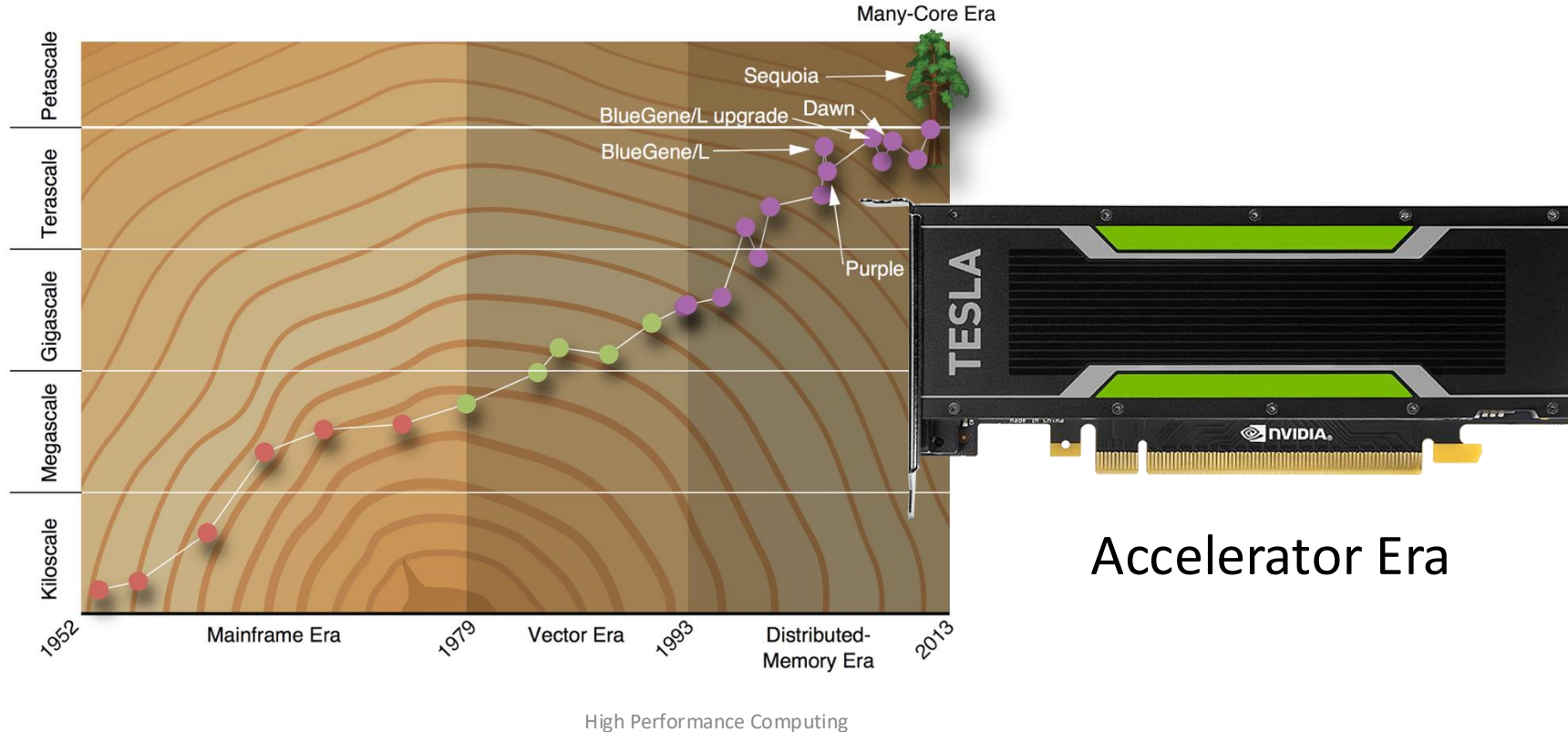


2000–2010
Multiple CPUs per node
sharing main memory



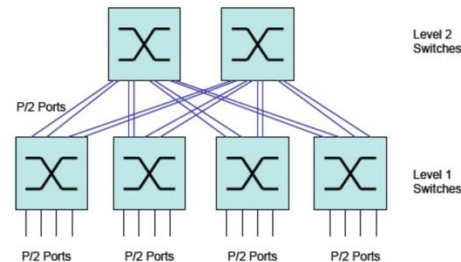
2000–2010
Accelerators usher in era
of heterogeneity

Evolution of supercomputers



Communication Network

- Connecting millions of processors requires a high-performance network
- Most existing systems use “Infiniband” or variations.
- Need a network **switch** to interconnect all the nodes
- Need high quality fiber **cables** to connect each node to the switch



- Fat-tree network: non-blocking network topology invented by Charles Clos (1953)
- Depending on how many switches you can afford, you might choose blocking configurations.
- <http://www.mellanox.com/clusterconfig/>
- <http://clusterdesign.org/fat-trees/>
- Key network parameters (switches and cables) are the **latency** and the **bandwidth**
- **Infiniband network**: latency 0.5-1 microsec, bandwidth 80-240 GB/sec
- Compare **Ethernet**: latency 20 microsec, bandwidth 0.1-40 GB/sec

Consequences

For Users

- It is necessary to exploit many (relatively slow) cores
- The memory per core is constant or even decreasing (memory limited)
- Higher level of parallelism
- I/O bottlenecks

For Developers

- Raw performance of individual core not increasing anymore
- More complex architecture (cache levels, accelerators, latency...)
- Multi-disciplinary approach is required, leading to the concept of “co-design”

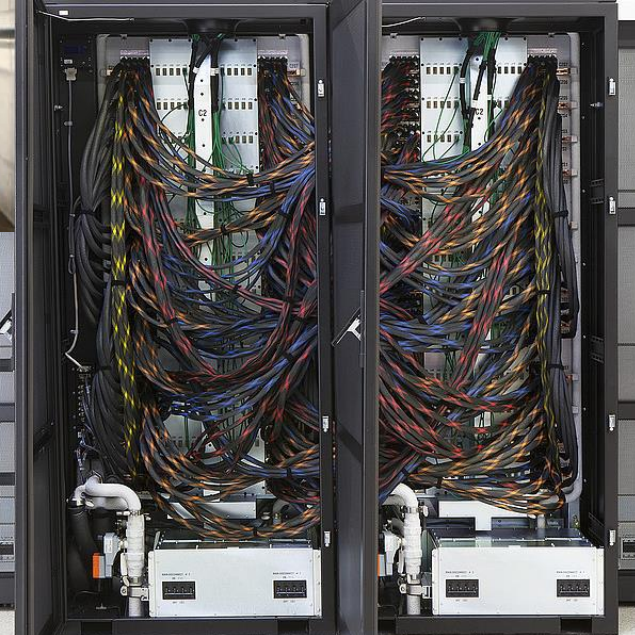
Programming Languages

Evolution of programming methods:

- MPI is still the dominant programming technique
- Hybrid OpenMP/MPI approach most effective on supercomputers
- GPU programming develops quickly (CUDA and OpenACC)
- Message Passing directly within the GPU
- New specific parallel programming languages are developed: Co-array Fortran, PGAS, X10, Chapel...
- New runtime systems to handle task-based parallelism: Charm++, HPX



Piz Daint (Alps) Architecture



Piz Daint Cooling

Alps Architecture

Model	HPE Cray EX
Performance	434.9 Petaflop peak performance 10 Exaflop BF16 performance
Interconnect	HPC Cray Slingshot-11 with 200 Gbps injection bandwidth per module / GPU
Scratch disk	100 + 10 PB on hard disk 5 + 1 PB on Solid State Disk (SSD)
Data archive	2 x 130 PB tape libraries

Nodes Overview

# of nodes	# of sockets per node	Total # of sockets	Processor(s)	Specifications	TFlops
2,688	4	10,752	NVIDIA Grace-Hopper	72 ARM cores, 128 GB LPDDR 5X RAM, H100 GPU with 96 GB HBM3 memory	n/a
1,024	2	2,048	AMD EPYC 7742 CPU (Rome)	2x64 cores, 256/512 GB DDR RAM	4,719
144	1 CPU + 4 GPU	720	AMD EPYC host CPU + NVIDIA A100	64 cores, 128 GB DDR RAM) and 4 NVIDIA A100-96/80 (96/80 GB HBM2E	n/a
128	4	512	AMD Mi300A CPU+GPU		n/a
24	1	24	AMD EPYC host CPU + AMD Mi250X GPU	64 cores, 128 GB RAM) and 4 AMD Mi250X GPU	n/a